AIST4010

Foundation of Applied Deep Learning

Assignment 1

Name: JIANG Huanyun

Student ID: 1155233165

1 Question 1 (10 pts)

Prove:

$$\frac{d}{dx}\,\sigma(x) \;=\; \sigma(x)\big(1-\sigma(x)\big), \qquad \sigma(x)=\frac{1}{1+e^{-x}}.$$

Write $\sigma(x)=\big(1+e^{-x}\big)^{-1}$; by the chain rule,

$$\frac{d\sigma}{dx} = -\big(1+e^{-x}\big)^{-2}\cdot\big(-e^{-x}\big) = \frac{e^{-x}}{\big(1+e^{-x}\big)^{2}}.$$

Split the result using $1+e^{-x}=\frac{1}{\sigma(x)}$:

$$\frac{e^{-x}}{\big(1+e^{-x}\big)^{2}} = \frac{1}{1+e^{-x}}\cdot\frac{e^{-x}}{1+e^{-x}} = \sigma(x)\left(1-\frac{1}{1+e^{-x}}\right) = \sigma(x)\big(1-\sigma(x)\big).$$

2 Question 2 (10 pts)

Prove: a multi-layer *linear* neural network is equivalent to a single-layer network.

L affine layers: $h^{(l)}=W^{(l)}h^{(l-1)}+b^{(l)}$, $h^{(0)}=x$. By induction on L. Base case $L=1$: trivial. If $h^{(L-1)}(x)=\hat{W}x+\hat{b}$ (induction hypothesis), then

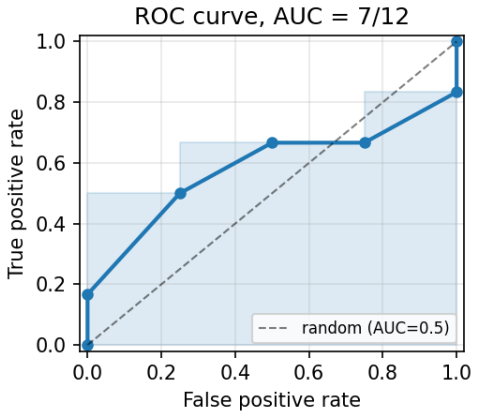
$$h^{(L)}(x) = W^{(L)}\big(\hat{W}x+\hat{b}\big)+b^{(L)} = \underbrace{W^{(L)}\hat{W}}_{\tilde{W}}\,x+\underbrace{W^{(L)}\hat{b}+b^{(L)}}_{\tilde{b}} = \tilde{W}x+\tilde{b},$$

which is again one affine layer. Hence every L-layer linear network reduces to a single affine layer; the function classes coincide.

3 Question 3 (10 pts)

Threshold classifier: $\hat{y}=1 \iff x\ge t$. $P=6$ positives, $N=4$ negatives.

Threshold t	TPR	FPR	ROC point
0.9	1/6	0	(0, 1/6)
0.7	1/2	1/4	(1/4, 1/2)
0.6	2/3	1/2	(1/2, 2/3)
0.4	2/3	3/4	(3/4, 2/3)
0.3	5/6	1	(1, 5/6)
0.1	1	1	(1, 1)



AUC (trapezoid sum over FPR):

$$\begin{aligned}\text{AUC} &= \tfrac{1}{2}\Big[\big(0+\tfrac{1}{6}\big)\cdot\tfrac{1}{4} + \big(\tfrac{1}{6}+\tfrac{1}{2}\big)\cdot\tfrac{1}{4} + \big(\tfrac{1}{2}+\tfrac{2}{3}\big)\cdot\tfrac{1}{4} + \big(\tfrac{2}{3}+\tfrac{2}{3}\big)\cdot\tfrac{1}{4} + \big(\tfrac{2}{3}+\tfrac{5}{6}\big)\cdot\tfrac{1}{4}\Big] \\ &= \frac{1}{2}\left[\frac{1}{24}+\frac{1}{6}+\frac{7}{24}+\frac{1}{3}+\frac{3}{8}\right] = \frac{7}{12} \approx 0.5833.\end{aligned}$$

4 Question 4 (20 pts)

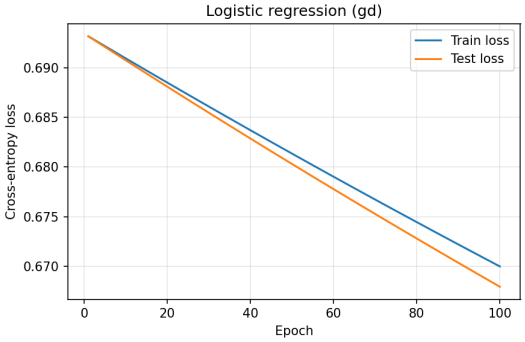
$lr = 0.001$, 100 epochs, init $W = \mathbf{0}$, $b = 0$. Loss = average binary cross-entropy. 2000 train samples (1000 per class), 100 test samples, 100 features, fixed seed.

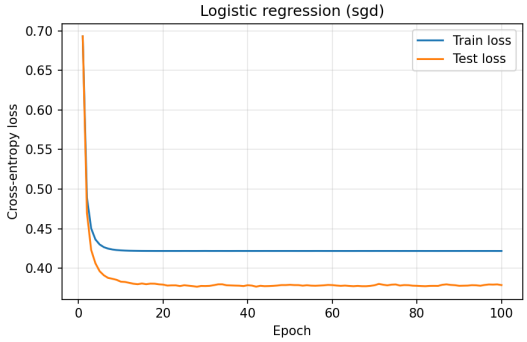
(a) Data: Gaussian inputs shifted to opposite sides of a random hyperplane; noise std 0.5 and 3% label flips ⇒ not perfectly separable. See `A1_Q4.py`.

(b) GD (NumPy). Accuracy = **0.84**. Loss: train 0.6931 → 0.6700, test 0.6931 → 0.6680. $\|W\|_2 = 0.0483$, $b = 0.0005$.

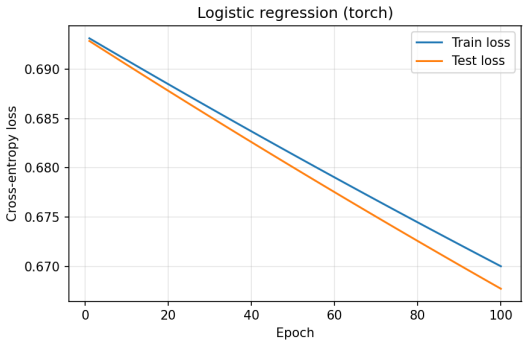
(c) SGD (NumPy). Accuracy = **0.84**. Loss: train 0.6931 → 0.4219, test 0.6931 → 0.3787. $\|W\|_2 = 1.5297$, $b = 0.0217$.

(d) Loss curves:





(e) PyTorch. `Linear(100,1)` + `Sigmoid`, `BCELoss`, `torch.optim.SGD`, no loop over samples. Accuracy = **0.84**. Loss: train 0.6931 → 0.6700, test 0.6929 → 0.6677. $\|W\|_2 = 0.0483$, $b = 0.0005$ (zero init ⇒ matches GD).



5 Question 5 (10 pts)

Iris, 105 samples, 5-fold stratified CV, standardized features.

Model	CV accuracy
Logistic regression	0.9524 ± 0.0426
SVM (RBF)	$\mathbf{0.9714 \pm 0.0381}$
FCN $4 \to 64 \to 32 \to 3$	0.9333 ± 0.0486
CNN (1-D)	0.9429 ± 0.0356

Best: SVM.

Why no deep-network advantage:

- Only 150 samples – too little data to estimate many parameters;
- Nearly linearly separable, shallow boundaries suffice – extra depth adds variance, not accuracy;
- 4 tabular features: no spatial/hierarchical structure for convolutions or deep feature hierarchies to exploit.

6 Question 6 (40 pts)

6.1 AlexNet (ImageNet Classification with Deep CNNs)

Motivation: hand-engineered features had plateaued; test whether a big, learned deep CNN could handle ImageNet's 1.2M images / 1000 classes. **Novelty:** first deep CNN to win ILSVRC by a large margin; ReLU, 2-GPU training, overlapping pooling, LRN, augmentation + dropout (60M params). **Intuition:** conv weight sharing + hierarchical features; ReLU gradients do not vanish, so depth becomes trainable. **Implementation:** `https://github.com/pytorch/vision` (AlexNet)

6.2 ResNet (Deep Residual Learning for Image Recognition)

Motivation: deeper plain nets had *higher training error* – optimization problem, not overfitting. **Novelty:** residual learning $H(x) = F(x) + x$ via skip connections; 152 layers, ILSVRC 2015 winner. **Intuition:** residuals near identity are easy to learn; skips give gradients a direct path, so extra depth only helps. **Implementation:** `https://github.com/pytorch/vision` (ResNet)

6.3 U-Net

Motivation: biomedical segmentation has tiny annotated datasets; sliding windows slow; upsampling lost detail. **Novelty:** symmetric encoder-decoder with skip concatenations + elastic augmentation; ISBI 2012/2015 winner. **Intuition:** encoder gives *what* (context), decoder recovers *where*; skips restore fine spatial detail. **Implementation:** `https://github.com/milesial/Pytorch-UNet`

6.4 SimCLR

Motivation: self-supervised features lagged supervised ones; prior contrastive methods needed memory banks / special architectures. **Novelty:** simple recipe: strong augmentation, small projection head, NT-Xent loss, large batches; matches supervised ResNet-50 on 1% labels. **Intuition:** attract augmentations of the same image, repel others $\Rightarrow$ learn nuisance-invariant semantics. **Implementation:** `https://github.com/google-research/simclr`

6.5 MAE (Masked Autoencoders)

Motivation: masked prediction dominated NLP but images are too redundant for naive reconstruction. **Novelty:** asymmetric ViT: 75% patches masked and removed; light decoder; 3-4× faster; ViT-H 87.8% ImageNet. **Intuition:** heavy masking forces semantics; discarding patches cuts compute; decoder thrown away after pre-training. **Implementation:** `https://github.com/facebookresearch/mae`